

Claude Code運用計画（クロコ統合版）

目的

認知的コストと実行性の乖離による生産コストの低下を補う装置

原動力は自身のような2Eやギフテッドと呼ばれる特性のある人のためだが組織でも導入が可能か？

考察メモ（気づきログ）

気づき→論点→仮説→検証/次アクション、の順で書くと整理しやすい。

「指示出しが重い」系の話は、意欲ではなく**“起動コスト（実行関門）”**として扱う。

書き込みテンプレ（コピペ用）

- 気づき（1行）：
- 背景/きっかけ：
- 論点（何が問題か）：
- 仮説（なぜ起きるか）：
- 設計原則（どう解くか）：
- 試すこと（次アクション）：
- メモ/参考：

2026-07-17：ゼロインタラクションとAI指示コスト

- 気づき（1行）：この運用思想は「ゼロインタラクション（摩擦を環境側で消す）」と同型。
- 論点: AIは“操作回数を減らす”一方で、要件化・言語化・評価・再指示の**インタラクション負債**を増やし得る。
- 重要: 特性によってはAIへの指示出し自体が物理的/認知的に高コストで、「生まれつきやる気がなくてできない」という主観的苦悩として現れることがある。
- 設計原則（案）：

- 指示の回数を減らす（1回で長く走る／定期実行／バッチ化）
- 指示の形を軽くする（選択式・定型ボタン・短文トリガー）
- 判断を減らす（テンプレ化・入力欄の制約）
- 失敗の痛みを減らす（ログ・再実行・途中保存）

2026-07-21：研究への発展の際の評価指標

- 気づき（1行）:定量的な評価や実務上の評価をしなければならない
 - 背景/きっかけ:Geminiとの対話
 - 論点（何が問題か）:研究では客観的な評価指標が必要であるから現在の私が便利だと感じるものだけではいけない
 - 設計原則（案）：
 -
 - 指示の形を軽くする（選択式・定型ボタン・短文トリガー）
-

全体像（3層アーキテクチャ）

1) スマホ（指示・閲覧の窓口）

- 役割: すべての指示入力 of 入口。外出先からも操作する。
- 主なツール: `Dify (PWA)`
- 閲覧: 予定は `Notion (カレンダー画面)` で確認

2) ラズパイ（常時起動ハブ：日常タスク+ログ蓄積）

- 役割:
 - 日常タスク（例: PDFリンク等からのスケジュール抽出→登録）を自動処理
 - プロジェクト開発用のブレストログ/仕様書（Markdown）を蓄積・同期
- 主なツール:
 - `Dify (OSS)`
 - `Gemini API`
 - `Docker / Docker Compose`
 - `Cloudflare Tunnel`（外部から安全にアクセス）

- `Syncthing` (PCへ暗号化同期)
- 方針 (クロコ3準拠) : メモリ節約のため、ラズパイ側での **Claude Code** 運用は除外する。

3) PC (本丸開発 : Claude Code 実行環境)

- 役割:
 - ラズパイから同期された指示書 (.md) を最優先コンテキストとして読み込み
 - 環境構築が必要な開発や複雑な実装をガリガリ進める
 - 主なツール:
 - `Claude Code (Anthropic公式CLI)`
 - `Node.js / npm`
-

データの流れ (ワークフロー)

A. 日常タスク (スケジュール管理など)

1. スマホ (Dify) : 「このPDF (URL) の予定をカレンダーに反映して」などを指示。
2. ラズパイ (Dify + Gemini) : URL/PDF等を解析し、予定データを構造化。
3. ラズパイ → Notion: Notion API経由で `Notionカレンダー` に予定を自動登録。
4. 結果: スマホで常に最新の予定を確認でき、ノートPCを開く頻度を減らす。

B. 本格開発プロジェクト (仕様書 → 実装)

1. スマホ (Dify) : 移動中にアイデア出し・壁打ち (ブレストログ生成)。
 2. ラズパイ (Dify + Gemini) : セッションを要約し、Markdownの「指示書/仕様書」としてローカル保存。
 3. 同期 (Syncthing) : ラズパイの出力フォルダをPCへ暗号化自動同期。
 4. PC (Claude Code) : 同期された .md を最優先コンテキストとして読み込み、実装を開始。
-

フォルダ設計 (ログ/成果物の分離)

クロコ1の構想を、クロコ3の運用（ラズパイはハブ、Claude CodeはPC）に合わせて整理。

例（SyncthingでPCにも同期する前提）：

- `workstation/`
 - `history/` ... スマホ→Dify→ラズパイで生成されたブレスト/仕様書ログ
 - `project-1_fix-notes.md`
 - `project-2_idea.md`
 - `output/` ... PC側（Claude Code等）で作った成果物・実行結果
 - `project-3_schedule.md`

狙い：

- スマホからも成果物を読める（outputを同期しておく）
- ブレストログ（history）と成果物（output）が混ざらず、再利用しやすい

セキュリティ/運用上の要点

- 外部アクセス: `Cloudflare Tunnel` を使い、ポート開放なしで安全にスマホ→ラズパイへアクセス。
- 同期: `Syncthing` でP2P暗号化同期（ラズパイ→PC）。
- エラー/制限への対処（クロコ1のメモ由来）：
 - APIのレート制限やエラー種別を正しく判定し、待機→再試行などのリトライ戦略を入れる。
 - 無駄な再試行を避ける（エラーの種類で分岐）。

構築ロードマップ（実施手順）

1. **基盤**: ラズパイにRaspberry Pi OS 64bitを導入し、Docker環境を構築。
2. **ハブ**: Docker上にDifyをデプロイ（メモリ消費を最適化）。
3. **通信**: Cloudflare Tunnelをセットアップし、外部からのHTTPSアクセスを確立。
4. **同期**: ラズパイとPCにSyncthingを導入し、`workstation/` を同期。

5. **日常タスク自動化:** GeminiでPDF等を解析 → Notionに予定登録するワークフローをDifyで構築。
 6. **開発環境:** PC側にNode.jsとClaude Codeを導入し、同期された .md をトリガーに運用テスト。
-

期待メリット（統合）

- **PCを開くハードルが下がる:** 日常の雑務はラズパイが処理し、PCは開発に集中できる。
- **安全性の担保:** Cloudflare Tunnelにより自宅サーバー公開リスクを抑える。
- **開発の再現性が上がる:** ブレスト → 仕様書（.md） → Claude Code実装、の流れが固定化される。